

Viva Questions (Experiment 1 to Experiment 4)

Experiment 1.1 (Post Composer Interface)

1. What is a Post Composer?
2. Why is a Post Composer used?
3. What is a User Interface (UI)?
4. What is React.js?
5. Why is React used for frontend development?
6. What is a Component in React?
7. What is JSX?
8. What is State in React?
9. What is `useState()`?
10. What is controlled input in React?
11. What is multi-platform content?
12. Why do different social media platforms have different constraints?
13. What is character count?
14. Why is character count important?
15. What is input validation?
16. What is real-time validation?
17. What is responsive design?
18. What is event handling in React?
19. What is `onChange()`?
20. What is conditional rendering?

-
1. How is platform selection implemented?
 2. How is character count updated dynamically?
 3. How do you display validation messages?
 4. What happens when the character limit is exceeded?
 5. How do you handle multiple platform selection?
 6. What are platform-specific validation rules?
 7. How can media validation be implemented?
 8. What is form validation?
 9. Difference between client-side and server-side validation?
 10. Why is real-time feedback important?
 11. What is component reusability?
 12. How is responsive UI achieved in React?
 13. What is a textarea?
 14. What is the difference between an input field and a textarea?
 15. What is dynamic rendering?

-
1. How would you support multiple social media platforms in one composer?
 2. How can platform constraints be stored efficiently?
 3. How can validation be optimized for performance?
 4. What challenges arise while designing a post composer?
 5. Explain the complete workflow of a multi-platform post composer.

Experiment 1.2 (Draft Management System)

1. What is a Draft?
 2. Why is a Draft Management System required?
 3. What is CRUD?
 4. What does Create operation do?
 5. What does Read operation do?
 6. What does Update operation do?
 7. What does Delete operation do?
 8. What is frontend state management?
 9. What is `useState()`?
 10. What is `useReducer()`?
 11. What is Local Storage?
 12. Why is Local Storage used?
 13. What is browser storage?
 14. What is asynchronous programming?
 15. What is `async/await`?
 16. What is a Mock API?
 17. Why are Mock APIs used?
 18. What is JSON Server?
 19. What is a loading state?
 20. What is user feedback in UI?
-

1. How are drafts stored in React?
 2. How do you retrieve saved drafts?
 3. How do you edit a draft?
 4. How do you delete a draft?
 5. How is Local Storage different from a database?
 6. What is persistent storage?
 7. What happens when the browser is refreshed?
 8. How does Local Storage improve user experience?
 9. What is state synchronization?
 10. What is data persistence?
 11. How do Mock APIs simulate backend communication?
 12. What is an API request?
 13. What is an API response?
 14. Why do we use asynchronous workflows?
 15. How do you display loading indicators?
-

1. How would you connect the draft system with a real backend?
2. What are the advantages of using Mock APIs during development?
3. How can draft data be secured?
4. What are the limitations of Local Storage?
5. Explain the complete workflow of a Draft Management System from draft creation to retrieval and deletion.

Experiment 2.1 – Redux Toolkit (State Management)

1. What is React?
 2. What is State in React?
 3. What is Global State?
 4. What is Local State?
 5. What is Redux?
 6. Why do we use Redux?
 7. What problems does Redux solve?
 8. What is Prop Drilling?
 9. How does Redux avoid Prop Drilling?
 10. What is Redux Toolkit?
 11. Why is Redux Toolkit preferred over traditional Redux?
 12. What is a Redux Store?
 13. What is `configureStore()`?
 14. What is a Slice?
 15. What is `createSlice()`?
 16. What is Initial State?
 17. What is an Action?
 18. What is a Reducer?
 19. What is Dispatch?
 20. What is `useDispatch()`?
 21. What is `useSelector()`?
 22. What is Provider?
 23. Why do we wrap the application with Provider?
 24. What are CRUD operations?
 25. How do you add data in Redux?
 26. How do you update data in Redux?
 27. How do you delete data in Redux?
 28. Can Redux work without React?
 29. What is Immutable State?
 30. Why should state not be modified directly?
 31. What is a Single Source of Truth?
 32. Explain the Redux data flow.
 33. Difference between `useState()` and Redux?
 34. Difference between Context API and Redux?
 35. What are Middleware?
 36. What is Redux DevTools?
 37. Advantages of Redux Toolkit?
 38. Limitations of Redux?
 39. Where is Redux mostly used?
 40. Explain the lifecycle of a Redux application.
-

Experiment 2.2 – Redux Selectors & Performance Optimization

1. What is a Selector?
 2. Why are Selectors used?
 3. What is Derived State?
 4. What is Memoization?
 5. What is `createSelector()`?
 6. What is Reselect?
 7. What is a Memoized Selector?
 8. Difference between a normal selector and a memoized selector?
 9. What is filtering?
 10. What is searching?
 11. What is sorting?
 12. Why is performance optimization required?
 13. What is unnecessary re-rendering?
 14. What causes component re-rendering?
 15. What is `useMemo()`?
 16. What is `useCallback()`?
 17. What is `React.memo()`?
 18. Difference between `useMemo()` and `React.memo()`?
 19. Difference between `useMemo()` and `useCallback()`?
 20. Difference between `createSelector()` and `useSelector()`?
 21. What is caching?
 22. What is expensive computation?
 23. What is rendering?
 24. What is virtual DOM?
 25. How does memoization improve performance?
 26. What happens if memoization is not used?
 27. What is component optimization?
 28. How do selectors improve performance?
 29. What is the purpose of derived data?
 30. Explain the complete flow of memoized selectors.
-

Experiment 3.1 – JWT Authentication

1. What is Authentication?
 2. What is Authorization?
 3. Difference between Authentication and Authorization?
 4. What is JWT?
 5. What is the full form of JWT?
 6. Why is JWT used?
 7. What are the three parts of JWT?
 8. What is Header?
 9. What is Payload?
 10. What is Signature?
 11. What information is stored in Payload?
 12. What is Stateless Authentication?
 13. Why is JWT called Stateless?
 14. What is Session-based Authentication?
 15. Difference between Session Authentication and JWT Authentication?
 16. What is Token-based Authentication?
 17. What is a Token?
 18. Where is JWT stored?
 19. What is Local Storage?
 20. Difference between Local Storage and Session Storage?
 21. How is JWT generated?
 22. How is JWT verified?
 23. What happens during Login?
 24. What happens during Logout?
 25. Why should JWT be protected?
 26. Can JWT expire?
 27. What is Token Expiration?
 28. What is Token Validation?
 29. What is Token Decoding?
 30. What is Base64 Encoding?
 31. What is the purpose of `atob()`?
 32. What is the purpose of `btoa()`?
 33. Advantages of JWT?
 34. Limitations of JWT?
 35. What happens if the token is modified?
 36. What happens if the token is missing?
 37. How is the Dashboard accessed after login?
 38. Explain the complete JWT authentication flow.
 39. What are the applications of JWT?
 40. Why is JWT preferred in modern web applications?
-

Experiment 3.2 – Role-Based Access Control (RBAC)

1. What is Role-Based Access Control (RBAC)?
 2. Why is RBAC used?
 3. What is a User Role?
 4. What are Permissions?
 5. Difference between Authentication and Authorization?
 6. What are the common user roles?
 7. What is an Admin?
 8. What is an Editor?
 9. What is a Viewer?
 10. What is Route Protection?
 11. What is a Protected Route?
 12. Why do we use React Router?
 13. What is BrowserRouter?
 14. What are Routes?
 15. What is Navigate?
 16. What is Unauthorized Access?
 17. What is the 403 Access Denied page?
 18. Why is route protection important?
 19. How does RBAC improve security?
 20. How are user roles stored?
 21. How are roles retrieved?
 22. What is the purpose of `ProtectedRoute`?
 23. What happens if a Viewer accesses the Admin page?
 24. What happens if the user is not logged in?
 25. How is Logout implemented?
 26. Difference between Public Route and Protected Route?
 27. Can one user have multiple roles?
 28. Advantages of RBAC?
 29. Limitations of RBAC?
 30. Explain the complete RBAC workflow.
-

Experiment 4.1 – Interactive Calendar Scheduling

1. What is an Interactive Calendar?
 2. Why are Calendar components used?
 3. What is Event Scheduling?
 4. What is Temporal Data?
 5. What is Event Mapping?
 6. What is FullCalendar?
 7. Why is FullCalendar used?
 8. What is Day View?
 9. What is Week View?
 10. What is Month View?
 11. What is Event Rendering?
 12. What is Dynamic Rendering?
 13. What is Drag-and-Drop Scheduling?
 14. What is Event Click?
 15. What is Event Resize?
 16. How are events stored?
 17. How are events displayed?
 18. What is Calendar State Management?
 19. What is Date Handling?
 20. What is Information Visualization?
 21. What is Human-Computer Interaction (HCI)?
 22. What are the advantages of calendar-based scheduling?
 23. What are the applications of calendar systems?
 24. Explain the working of an interactive calendar.
 25. What libraries are used for calendar implementation?
-

Experiment 4.2 – Performance Optimization & Testing

1. What is Performance Optimization?
2. Why is optimization required?
3. What is Rendering?
4. What is Re-rendering?
5. What is unnecessary re-rendering?
6. What is Memoization?
7. What is `React.memo()`?
8. What is `useMemo()`?
9. What is `useCallback()`?
10. Difference between `React.memo()` and `useMemo()`?
11. Difference between `useMemo()` and `useCallback()`?
12. What is Component Optimization?
13. What is Expensive Computation?
14. What is Stable Function Reference?
15. What is State Optimization?
16. What is Component Testing?
17. What is Unit Testing?
18. What is Integration Testing?
19. What is Functional Testing?
20. What is Jest?
21. What is React Testing Library?
22. Why is testing important?
23. What is a Test Case?
24. What is `render()` in React Testing Library?
25. What is `screen.getByText()`?
26. What is `expect()`?
27. What is `toBeInTheDocument()`?
28. What is Browser DevTools?
29. How do you identify performance bottlenecks?
30. Explain the complete optimization and testing workflow.